

第十四讲：通用机器人策略

Generalist Robot Policies: 流匹配、VLA 与机器人基础模型

马漫游

深圳技术大学人工智能学院

2026 年 6 月 27 日





本讲大纲

1 流匹配 Flow Matching

上一讲扩散模型的连续、确定性推广—— π_0 的动作生成引擎

2 从专才到通才

预训练-再适配范式: RT-1/RT-2、Open-X、OpenVLA 与数据里程碑

3 VLA: 视觉-语言-动作模型

现代配方解剖: VLM 骨干 + 生成式动作专家 + 动作分块

4 π_0 与 SmolVLA

两个代表模型的架构、训练与代码

5 规模、显存与前沿

模型多大、GPU 要多少显存、2025-26 最新模型一览

本讲核心观点

通用机器人策略 = 互联网级 VLM 先验 + 生成式动作头 (流匹配) + 大规模跨形态数据——三者缺一不可。



第 1 节

流匹配 Flow Matching



上讲回顾：扩散模型的成与不足

扩散模型做对了什么

- ▶ 固定的高斯后验 $q(z_t | z_{t-1}) = \mathcal{N}(\sqrt{1 - \beta_t} z_{t-1}, \beta_t \mathbf{I})$
⇒ 似然也是高斯，一切闭式可推

- ▶ 训练目标简单到极致：**预测噪声**

$$\mathcal{L}(\theta) = \mathbb{E}[\|\epsilon - \epsilon_\theta(\sqrt{\alpha_t} z_0 + \epsilon \sqrt{1 - \alpha_t}, t)\|^2]$$

- ▶ 能拟合高度多模态的分布 ⇒ Diffusion Policy 的成功

但推理是个负担

- ▶ 去噪是**随机、离散、多步**的：可能要**上百步**昂贵的网络前传才能恢复一个样本
- ▶ 加噪 / 去噪本质是**布朗运动**：路径来回**折返**，浪费计算
- ▶ 机器人控制是**实时的**——动作生成越快越好

本节的问题

能不能保留「从噪声变换出复杂分布」的能力，但把路径修直、把步数减少？

⇒ **流匹配** (Flow Matching, Lipman et al. 2023)

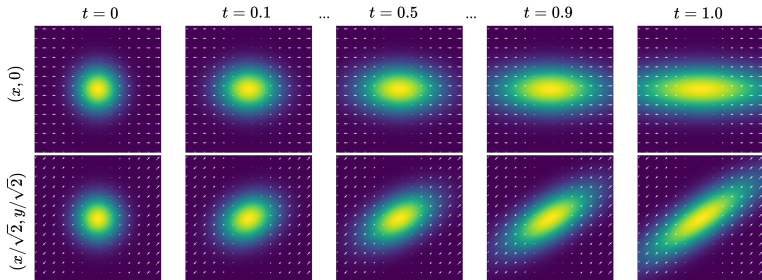
FM 是 DM 的**连续时间、确定性**推广——也是 π_0 、GR00T、SmoI VLA 共同选择的动作生成引擎。



核心思想：用向量场「运输」概率质量

FM 学习一个**确定性、连续、可微**的流 $\psi: [0, 1] \times Z \mapsto Z$, 由 (可随时间变化的) **向量场** $v: [0, 1] \times Z \mapsto Z$ 驱动, 把简单先验 p_0 (如标准高斯) 的样本**随时间运输**到复杂的数据分布 p_1 :

$$\frac{d}{dt} \psi(z, t) = v(t, \psi(t, z)), \quad \psi(0, z) = z$$



- ▶ 上下两行：同一初始分布，施加**不同向量场** $u_1(x, y) = (x, 0)$ 与 $u_2(x, y) = (x/\sqrt{2}, y/\sqrt{2})$, 质量沿**不同的流**演化——时间在 $[0, 1]$ 上**连续**流动
- ▶ FM 模型学的就是逼近一个目标向量场 u : $v_\theta \approx u \Rightarrow$ 诱导的流 ψ 逼近理想轨迹 ψ^*



π_0 用的流匹配：直线插值、匀速场

抛开一般理论， π_0 用的是 FM 最简单的形态，**三步就能定义**：

- ▶ **配对**：抽一个噪声 $z_0 \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ ，抽一个数据 $z_1 \sim \mathcal{D}$
- ▶ **连线**：在两点之间画**直线**： $z_t = (1 - t)z_0 + tz_1$ ， $t \in [0, 1]$
- ▶ **求速度**：对 t 求导—— $\frac{dz_t}{dt} = z_1 - z_0$ ，**全程恒定（匀速直线）**

于是训练就是普通的监督回归（这套做法叫**条件流匹配，CFM**）：

$$\mathcal{L}(\theta) = \mathbb{E}_{t, z_0, z_1} \left[\left\| v_{\theta} \left(\underbrace{(1 - t)z_0 + tz_1}_{\text{线上一点}}, t \right) - \underbrace{(z_1 - z_0)}_{\text{这条线的速度}} \right\|^2 \right], \quad t \sim \mathcal{U}([0, 1])$$

训练循环（每一步）

抽 $(z_0, z_1, t) \rightarrow$ 算线上点 $z_t \rightarrow$ 网络预测速度 $\rightarrow$ 对 MSE 反向传播。**没有 ELBO、没有 KL、不用积分 ODE。**

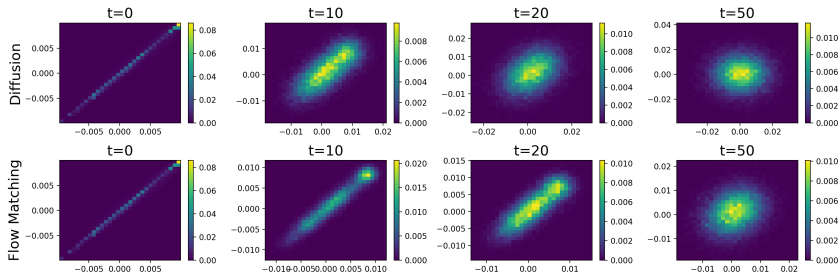
一个自然的疑问

逐样本回归「自己那条线」的速度，凭什么能学出把**整个**噪声分布变成数据分布的场？——Lipman 等（2023）证明二者**梯度相同**，放心用；直觉看后面的手算例子。



眼见为实：直线路径 vs 扩散路径

同一份机器人 (o, a) 联合分布、同样 $T=50$ 步——上排是扩散加噪，下排是直线流匹配：



- ▶ 上排 DM：布朗运动式加噪，路径杂乱、来回折返
- ▶ 下排 FM：源分布到目标分布之间清晰、笔直的插值
- ▶ 路径更直 $\Rightarrow$ 训练更快更稳；推理无需回溯 $\Rightarrow$ 少得多的步数就有同等质量



一个能手算的例子

取一维直线路径。**条件层面**：抽噪声 $z_0 = -1$ 、数据 $z_1 = 2$ ：

$$z_t = (1 - t) \cdot (-1) + t \cdot 2 = 3t - 1, \quad u(t, z_t | z_1) = z_1 - z_0 = 3 \quad (\text{全程匀速})$$

比如 $t=0.5$ ：训练样本就是「在 $(t, z) = (0.5, 0.5)$ 处把 v_θ 回归到 3」——一条普通的监督回归数据。

边缘层面（全高斯，可闭式验证）：设 $p_0 = \mathcal{N}(0, 1)$ 、 $p_1 = \mathcal{N}(2, 1)$ ，独立配对 (z_0, z_1) ：

$$z_t \sim \mathcal{N}(2t, (1 - t)^2 + t^2), \quad u(t, z) = \mathbb{E}[z_1 - z_0 | z_t = z] = 2 + \frac{2t - 1}{(1 - t)^2 + t^2} (z - 2t)$$

- $t=0$ ： $u = 2 - z$ ——一边收缩一边奔向数据均值； $t=\frac{1}{2}$ ： $u \equiv 2$ （常数！「腰部」处所有直线速度的后验平均恰好相同）；网络学的正是 $\mathbb{E}[z_1 - z_0 | z_t = z]$ ：**穿过 (t, z) 的所有直线速度的后验加权平均**

这也是「平均」，为什么不怕模态平均？

第十二讲的模态平均，平均的是**最终动作**；这里平均的只是**瞬时速度**：ODE 随时间推进，轨迹会**分岔流向不同模态**，终态分布精确等于 p_1 ——多模态完好保留。



与扩散面对面比较

把前面定义的直线流放到第十三讲的扩散模型旁边，逐项对比：

	扩散模型 DM (第十三讲)	流匹配 FM (本讲)
时间	离散 $t \in \{1, \dots, T\}$	连续 $t \sim \mathcal{U}([0, 1])$
路径	随机 (布朗运动, 折返)	确定性 (直线插值)
加噪方式	高斯逐步扰动	两点连线 (无需高斯推导)
训练目标	预测噪声 ϵ	回归向量场 $v_\theta \rightarrow (z_1 - z_0)$
推理	迭代去噪, 数十 ~ 数百步	ODE 前向积分, ~10 步 即可

推理 = 解一个 ODE

从 $z_0 \sim p_0$ 出发, 沿 $\frac{dz}{dt} = v_\theta(z_t, t)$ 积分到 $t = 1$ ——实践中用**前向 Euler** 跑十来步即可。对**实时机器人控制**, 这正是 FM 相对 DM 的决定性优势 (π_0 与 SmolVLA 都只用 **10 步**)。

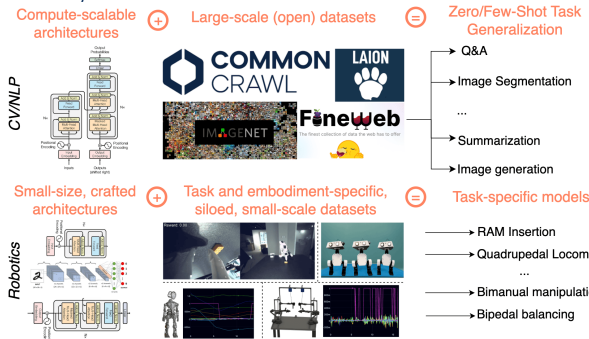


第 2 节

从专才到通才



预训练-再适配: CV / NLP 的剧本能否复制?



上: CV / NLP 的基础模型范式

- ▶ (1) 任务无关的**大规模预训练** + (2) 任务专属的**轻量适配**
- ▶ 靠两条腿走路: **架构创新** (Transformer 捕捉长程依赖、稳定可扩展) 与 **数据-算力联合扩展**

下: 机器人学的「碎片化」现状

- ▶ 架构不统一 + 数据孤岛 (小型任务专属数据集)
- ▶ $\Rightarrow$ 长期只有**专才策略**: 在指定任务上很强, 换个部署场景就失效

第十二、十三讲的 ACT / Diffusion Policy 都是**单任务策略**; 本讲: 能跨任务、跨形态、听懂自然语言指令的**通才策略**。

机器人数据为什么难以「堆」起来？



与互联网上唾手可得的任务无关文本 / 图像不同，机器人数据**天生具身 (embodied)**：

方法论挑战

- ▶ 每条专家轨迹绑定**特定平台 + 特定环境 + 特定任务**：操作数据 $\neq$ 移动数据
- ▶ 专家示范**采集昂贵**，且出了名地**异质**——不同人做同一任务方式迥异

概念性挑战

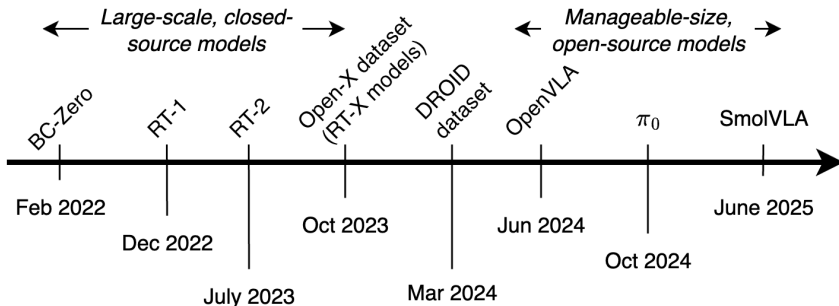
- ▶ 朴素地混合跨形态数据可能引发**负迁移 (negative transfer)**
- ▶ 为不同机器人独立开发的控制策略，合在一起甚至会**相互冲突**

但转机出现了

Open X-Embodiment 实验证明：**单个模型在多形态数据上训练，可以胜过各自数据上的专才模型**——正迁移是真实存在的。剩下的问题是把数据与配方做对。



通才策略的时间线



- ▶ **闭源先行：**BC-Zero（隐变量模型，25k+ 示范）→ RT-1 → RT-2（「VLA」一词的来源）
- ▶ **开放数据集解锁开源模型：**Open-X 与 DROID 发布后，OpenVLA、 π_0 、SmoIVLA 相继出现
- ▶ 从 BC-Zero 的 2.5 万条示范，到 π_0 的 **1000 万 + 条轨迹**与强 few-shot 能力——五年走完

下面挑三个里程碑细看：RT-1 / RT-2（范式确立）、Open-X / DROID（数据基建）、OpenVLA（开源平权）。

RT-1 → RT-2: 把控制变成「语言建模」



RT-1 (2022): 规模化的开始

- ▶ 13 万条人类示范·13 台机器人·历时 17 个月
- ▶ 输入: 图像历史 + 自然语言指令 → Transformer
- ▶ 输出: **离散化动作**——6 关节各分 256 档, 用**分类损失**训练
- ▶ 证明了「单个 Transformer 吞下大规模多任务示范」可行

RT-2 (2023): VLA 的诞生

- ▶ 把控制写成**视觉问答 (VQ&A)**: 文本 token 与 8-bit **动作 token** ($2^8=256$ 档) 共享同一输出空间
- ▶ 在 (1) 网络数据 + (2) 机器人数据上**联合微调** PaLI-X / PaLM-E 等大 VLM
- ▶ 继承互联网级**语义知识**: 从未学过「锤东西」, 却能在纸 / 耳机 / **石头**中正确选出能当锤子的

范式确立

Vision-Language-Action: 用预训练 VLM 当大脑, 把动作并入 token 词表——代价是动作仍是**离散**的, 高频连续控制吃力 ⇒ 第 3 节现代 VLA 改用**生成式动作头**。

数据基建与开源平权



两大数据集里程碑

- ▶ **Open X-Embodiment** (2023): 22 种形态·21 家机构·60 个既有数据集的聚合 $\Rightarrow$ **140 万条** 开放轨迹; 并实证**跨任务跨形态正迁移**
- ▶ **DROID** (2024): **7.5 万 +** 条真实 (in-the-wild) 操作示范——通才策略的另一块基石
- ▶ 中心化大集合之外: LeRobot 等库让**去中心化社区贡献**成为常态 (第十一讲)

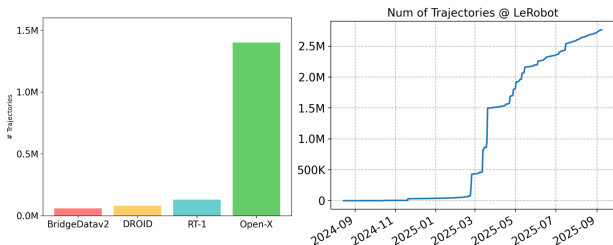
OpenVLA (2024): 对抗「可及性鸿沟」

- ▶ RT-1 / RT-2 是**闭源**的, 训练这类模型的算力多数机构难以企及
- ▶ OpenVLA **只用开放数据** (Open-X 的 97 万条轨迹), 开源**权重 + 全套训练配方**
- ▶ 架构: 预训练视觉编码器 $\rightarrow$ 投到 **Llama2-7B** 嵌入空间 $\rightarrow$ 预测 256 档**离散动作 token**
- ▶ 7B 模型性能超过 55B 的 RT-2-X——**配方与数据 > 纯堆参数**

开放数据 + 开放配方, 把通才策略研究从「巨头特权」变成了「社区事业」。



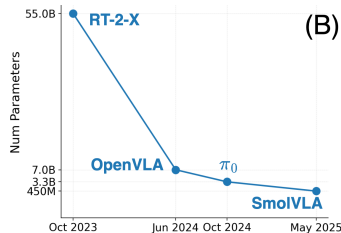
两条同时发生的趋势曲线



(A)

(A) 数据：更大，且更去中心化

- ▶ 中心化集合屡创新高 (Open-X 140 万条)
- ▶ LeRobot 社区数据一年冲到 270 万 + 条轨迹——**去中心化**正在追平学术基准



(B)

(B) 模型：性能不减，体积锐减

- ▶ RT-2-X 55B $\rightarrow$ OpenVLA 7B $\rightarrow \pi_0$ 3.3B $\rightarrow$ SmolVLA 450M
- ▶ 越来越容易跑在**真实、资源受限**的机器人上 (第 5 节算给你看)



第 3 节

VLA: 视觉-语言-动作模型



现代 VLA 的标准配方

现代 VLA (π_0 、SmolVLA.....) 在 RT-2 范式上做了两类关键升级：

架构：统一 Transformer + 分工专家

单个 Transformer 保证**计算效率**，内部用**不相交的权重集 (experts)** 分别负责视觉-语义理解与控制——类似 MoE，部署时更省算力。语言条件化保证跨任务能力。

流程：连续动作 + 动作分块

(1) 中等规模**动作专家**直接建模**连续分布** $p(a_{t:t+H_a} | o_t)$ ——彻底抛弃离散动作 token；(2) 沿用 **action chunking** (第十二讲) 抑制非 i.i.d. 数据上的误差复合。

相对任务专属方法的三大收益：

- ▶ **白嫖世界知识**：互联网级预训练骨干自带视觉 / 语言 / 语义概念，不必从零学
- ▶ **生成式动作头**：大数据下示范几乎必然**多模态** (第十二讲的教训) ——生成模型是刚需
- ▶ **感知与动作分离**：MoE 式结构推理更高效——真实部署的关键属性

成果：端到端叠衣服、收拾餐桌这类高灵巧长程任务，few-shot 适配新任务。



背景知识：VLM 是怎么炼成的

标准构造

- ▶ **预训练视觉编码器** (CLIP / SigLIP / DINOv2) + **预训练 LLM** (Llama / Mistral / Gemma) 拼接而成
- ▶ 多阶段多模态训练：先在**图文对**与图文交错语料上大规模预训练，再用**指令微调**数据做 SFT
- ▶ 结果：在视觉与语言上**联合推理**——无需逐概念监督就掌握物体、属性、关系

对 VLA 意味着什么

- ▶ VLA 把 VLM 当**感知骨干**，直接**继承**上下文世界知识
- ▶ 高层自然语言指令得以**落地 (ground)** 到机器人的视觉情境；没见过的物体也能挂到预训练概念上 $\Rightarrow$ 泛化

与机器人最相关的 VLM 趋势：小型化

用更小更多样的数据、更小的模型 (SmolVLM、Moondream、MiniCPM-V)、或只调少量参数适配单模态模型——**算力效率**已成多模态研究的核心议题。

为什么机器人格外在意

策略要在**真实世界的严苛约束** (机载算力、功耗、延迟) 下运行；视频 / 音频等更多模态的接入对多传感器机器人尤其有吸引力。

SmolVLA 的骨干 SmolVLM-2 正是这股小型化潮流的产物。

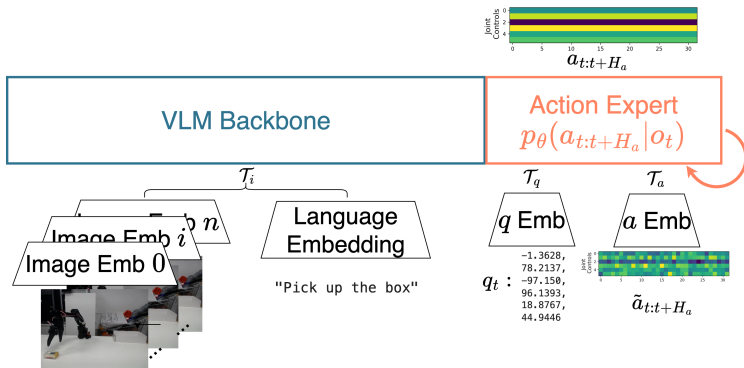


第 4 节

π_0 与 SmolVLA



π_0 总览: VLM 骨干 + 流匹配动作专家

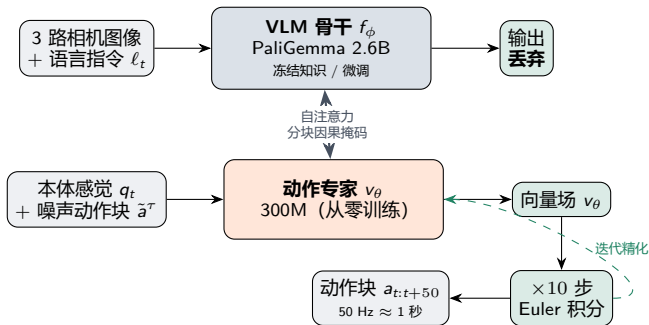


- ▶ 单个统一 Transformer, 两套不相交权重 ϕ, θ (MoE)
- ▶ VLM 骨干 f_ϕ : PaliGemma (Gemma 2.6B, late-fusion), 处理多路相机帧 $\{l_t\}$ 与语言指令 ℓ_t
- ▶ 动作专家 v_θ (300M, 从零初始化): 处理本体感觉 q_t 与动作块 $a_{t:t+H_a}$

- ▶ 两个专家各自生成 Q/K/V, 只通过自注意力层交换信息——保持分工, 又共享语境
- ▶ VLM 侧的输出弃用; 动作专家回归的向量场用于迭代精化动作块——正是第 1 节的流匹配
- ▶ 训练数据: 1000 万 + 条轨迹 (「 π 数据集」)



π_0 结构图：双专家统一 Transformer



- ▶ 两条输入通道：视觉-语言走 VLM；机器人特有量（本体 + 动作）走动作专家
- ▶ 一个 Transformer，两套权重 (ϕ, θ) ：仅经自注意力交换信息

- ▶ VLM 侧输出**丢弃**，只取动作专家回归的**向量场**
- ▶ 10 步 Euler 中**只重跑 300M 专家**；VLM 前缀 K/V **缓存复用**



π_0 的数据流：一次前向经过了什么

跟着 token 走一遍（规格： $H_a=50$ 步动作块 @ 50 Hz $\approx$ 1 秒动作）：

1. **图像**：3 路相机帧过 PaliGemma 的视觉编码器 $\rightarrow$ 视觉 token；**语言指令** $\ell_t \rightarrow$ 文本 token——两者在 VLM 主干内**late-fusion**（深层才融合）
2. **本体感觉** q_t ：线性投影成 1 个 token，走**动作专家**一侧的权重
3. **噪声动作块** $\tilde{a}_{t:t+H_a}$ ：每步动作嵌入 + 流时刻 τ 的正弦编码 $\rightarrow H_a$ 个动作 token，同样走动作专家
4. **同一个 Transformer 前向**：每层里两套权重 (ϕ, θ) 各自算 Q/K/V，唯一的信息交换发生在**自注意力**里（按下一页的分块因果掩码）
5. **输出**：VLM 侧 token 的输出**全部丢弃**；动作 token 的输出就是向量场 v_θ ——训练时进流匹配损失，推理时做积分

为什么 q_t 也走动作专家一侧？

本体感觉与动作**同处关节空间**，且对「只见过互联网图文」的预训练 VLM 来说是**分布外**的输入——把机器人特有的量都归到从零训练的专家一侧，让 VLM 专心做它擅长的视觉-语言理解。



π_0 的分块因果注意力

token 分成三块：图像-语言 $\mathcal{T}_i$ 、本体感觉 $\mathcal{T}_q$ 、动作 $\mathcal{T}_a$ (流过程时刻 τ 的噪声块 $a_{t:t+H_a}^\tau$)。块内双向注意力，块间只许向前看：

$$\mathbf{A} = \begin{array}{c|ccc} & \mathcal{T}_i & \mathcal{T}_q & \mathcal{T}_a \\ \hline \mathcal{T}_i & 1 & 0 & 0 \\ \mathcal{T}_q & 1 & 1 & 0 \\ \mathcal{T}_a & 1 & 1 & 1 \end{array}$$

1 : 双向注意, 0 : 屏蔽

动机 1: 保护 VLM 先验

机器人 token 对「在互联网数据上预训练」的 VLM 来说大概率**分布外**——屏蔽掉，防止污染感知-语言表征。

后续工作 (knowledge insulation) 进一步表明：不隔离 VLM 知识、任由流匹配梯度回传，确实**损害**性能。

动机 2: 推理时缓存 K/V

块间通信被屏蔽 $\Rightarrow$ 图像-语言块的 K/V 在**多步去噪间不变**，可整体**缓存复用**——显著降低每步去噪的计算量，加快推理。

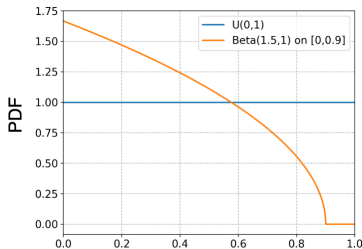


π_0 的训练：条件流匹配损失

对动作块做线性加噪 $\tilde{a}_{t:t+H_a} = \tau a_{t:t+H_a} + (1 - \tau) \epsilon$, 回归目标向量场 $(\epsilon - a_{t:t+H_a})$:

$$\mathcal{L}(\phi, \theta) = \mathbb{E}_{\tau, \epsilon, o_t, a} \left[\left\| v_{\theta}(\tilde{a}_{t:t+H_a}, o_t, \tau) - (\epsilon - a_{t:t+H_a}) \right\|^2 \right], \quad \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

- ▶ 与第 1 节的 CFM 目标同构——只是条件化到 o_t , 且作用在动作块上
- ▶ 每个动作 token 嵌入噪声动作 a_i^{τ} + 流时刻 τ 的正弦编码
- ▶ τ 不从均匀分布采! $\tau \sim \text{Beta}_{[0,s]}(1.5, 1)$: 偏向高噪声段——逼模型先学会重建条件均值 $\mathbb{E}[a | o]$, 而非恒等映射
- ▶ 支撑截断到 $[0, s]$, $s < 1$: 推理步长 $\delta = 1 - s$ 时, $\tau > s$ 反正永远用不到
- ▶ 推理: 前向 Euler 积分 $a^{\tau+\delta} = a^{\tau} + \delta v_{\theta}(a^{\tau}, o_t)$, 仅 10 步



修改后的 τ 采样分布：偏向更噪的时间步（截断 Beta）



π_0 的推理：10 步 Euler 的完整循环

推理算法（生成一个动作块）

1. 取观测 o_t (3 相机 + 指令 + q_t)，VLM 前缀**只前向一次**，缓存其 K/V
2. 初始化 $a_{t:t+H_a}^{\tau=0} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ (纯噪声动作块)
3. 对 $\tau = 0, \delta, 2\delta, \dots, 1 - \delta$ ($\delta=0.1$, 共 **10 步**): 只重跑**动作专家** (复用缓存 K/V), 前向 Euler 一步

$$a_{t:t+H_a}^{\tau+\delta} = a_{t:t+H_a}^{\tau} + \delta v_{\theta}(a_{t:t+H_a}^{\tau}, o_t, \tau)$$

4. 输出 $H_a=50$ 步动作块 (50 Hz $\approx$ 1 秒动作), 交给执行端

算力都花在哪

- ▶ 贵的部分 (VLM 读 3 张图 + 指令) **只做一次**, 被 K/V 缓存摊销
- ▶ 10 步循环里每步只有**小得多的 300M 动作专家**在跑
- ▶ 这正是分块因果掩码「顺带」买到的推理红利 (上一页动机 2)

与第十三讲 DP 的推理对比

DP: $T=100$ 步 DDPM, 每步整个 U-Net 前向、路径随机; π_0 : **10 步确定性 ODE 积分** + 大头计算缓存——同为「迭代精化」, 账单差出一个量级。



π 数据集：预训练 + 微调为什么赢

数据构成

- ▶ 1000 万 + 轨迹 = 私有遥操作数据 + Open-X + DROID; 开放部分仅占 $\approx 9.1\%$
- ▶ 灵巧示范覆盖 7 种机器人配置、68 类任务
- ▶ 跨形态处理：动作向量按全数据集最大配置定长，自由度少的机器人零填充；固定 3 路相机，缺失视角用掩码图像槽

核心实验发现

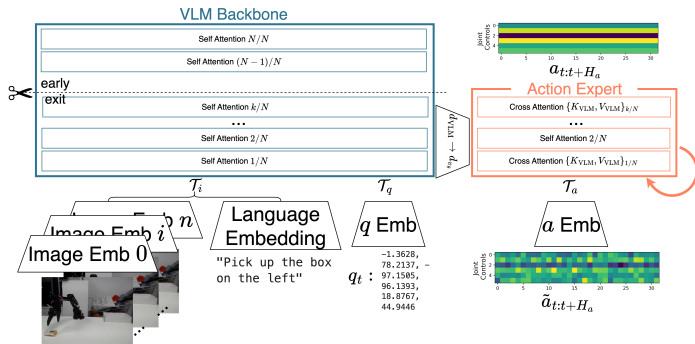
- ▶ π 预训练 + 任务微调全面胜过同任务从零训练的 π_0^{scratch}
- ▶ 直觉：高质量示范不含失败，纯 BC 学不会从濒临失败中恢复；大规模杂数据单看次优，却提供了恢复行为的「养分」
- ▶ 微调后获得流畅的多阶段长程行为：端到端叠衣服、收拾餐桌

这正是「预训练-再适配」范式在机器人上的落地

广而杂的预训练给鲁棒性，窄而精的微调给流畅度——与 LLM 的「预训练 + SFT」如出一辙。



SmolVLA: 450M 的「人人可跑」VLA

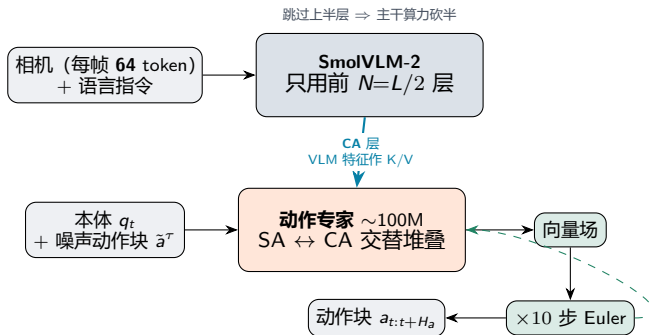


- ▶ 动机: π_0 仍然 (1) 大而吃算力 (2) 依赖昂贵平台上的闭源数据——SmolVLA 完全**开源模型 + 数据 + 配方**
- ▶ 同款 MoE + 流匹配设计; 骨干换成 **SmolVLM-2** (SigLIP 视觉编码器 + SmolLM2 语言解码器)
- ▶ 动作专家仅 $\sim 100M$, 嵌入维度压到 $d_{v_g} = 0.75 d_{VLM} \Rightarrow$ 总计 **450M** vs π_0 3.3B

- ▶ 与 π_0 不同点 1: 放弃分块因果掩码, 用**简单因果掩码**
- ▶ 不同点 2: 动作专家**交替堆叠**交叉注意力 (CA) 与自注意力 (SA) 层——CA 用动作 token 当 query、VLM 特征当 K/V; 实测**成功率更高、动作更平滑**
- ▶ K/V 同样可缓存 $\Rightarrow$ 推理提速



SmolVLA 结构图：更瘦的骨干 + CA/SA 专家



- ▶ **骨干瘦身**: SmolVLM-2 只取前 $L/2$ 层; 每帧视觉 token 压到**固定 64 个**
- ▶ **动作专家**嵌入维 $d_{v_\theta} = 0.75 d_{\text{VLM}}$, $\sim 100\text{M}$

- ▶ 与 π_0 的**关键不同**: 动作专家内**交替** SA (块内自治) 与 CA (向 VLM 取料)
- ▶ CA 的 K/V 同样**可缓存** $\Rightarrow$ 推理更快



SmolVLA 的动作专家：交替 CA / SA 层

π_0 的两个专家只靠（分块因果）**自注意力**交流；SmolVLA 把「读观测」和「块内协调」拆成两种层**交替堆叠**：

CA 层：向 VLM「取料」

- ▶ **query**：只有动作 token
- ▶ **key / value**：VLM 的视觉、语言、本体 token 投影到共享空间后提供
- ▶ K/V 在整个去噪过程中**可缓存**——推理提速

SA 层：块内自治

- ▶ 动作 token **之间**做因果自注意力（简单因果掩码，不用 π_0 的分块版）
- ▶ 负责让 H_a 步动作**内部协调**——平滑、不自相矛盾

两个实测结论

交替 CA/SA 比只用一种注意力**成功率更高、动作块更平滑**；动作专家嵌入维度压到 $d_{v\theta} = 0.75 d_{vLM}$ （共 $\sim 100M$ 参数）——表达力够用，算力省下来。



SmolVLA 的效率工程：三板斧砍计算

三板斧

- ▶ **视觉 token 瘦身**：像素重排 (pixel shuffle) 压到每帧**固定 64** $\uparrow$ token，弃用 VLM 预训练时的切片放大
- ▶ **跳过 VLM 上半层**：只取前 $N=L/2$ 层解码器特征——速度 / 性能的甜点，主干算力**砍半**
- ▶ **异步推理栈**：动作**预测与执行**解耦（第十三讲「推理优化」一节的方案：队列、阈值 g 、相似度过滤），普通硬件也能保持响应

省出来的算力意味着什么

- ▶ 推理同样是 **10 步流积分**（与 π_0 一致）
- ▶ **CPU / MacBook** 也能跑推理；单张消费级 GPU 即可微调（第 5 节细算）

成绩单

真实与仿真环境上**比肩** π_0 ，同时快 $\approx 40\%$ 、内存省 **6~7 倍**——在 SO-100/101 这类低成本平台上即可复现。

只用社区数据：把「野生数据」洗成预训练语料



π_0 的底气是 1000 万 + 条轨迹（九成私有）；SmolVLA 反其道而行——**只用完全开放的社区数据**：

数据从哪来

- ▶ 450+ 个 LeRobot 社区数据集，共 2 万 + 条轨迹（正是第十一讲 LeRobotDataset 生态的产出）
- ▶ 采集平台以 SO-100 / SO-101 为主——低成本平台把「采数据」平民化的直接红利

野生数据的两大脏处与清洗

- ▶ **任务描述缺失 / 噪声**（社区随手写的标注） $\Rightarrow$ 用现成小 VLM 看采样帧，**重新生成任务描述**
- ▶ **相机命名与视角混乱** $\Rightarrow$ 统一映射到 top / wrist / side 的固定顺序

这页的意义

预训练数据量差约 500 倍（2 万 vs 1000 万），性能却能比肩——**配方（架构 + 数据清洗）对了，开放小数据也能训出能打的 VLA**。「去中心化数据」路线（第 2 节趋势图 A 右）由此第一次被端到端验证。



算力账本: π_0 与 SmolVLA 各要多少

π_0 (3.3B)

- ▶ **参数:** VLM 骨干 $\approx 3\text{B}$ (SigLIP 视觉 + Gemma 2.6B) + 动作专家 300M
- ▶ **权重** (bf16) $\approx 6.6\text{ GB}$; 推理峰值 **$> 8\text{ GB}$** (3 路相机激活是大头, + VLM 前缀 K/V 缓存)
- ▶ **LoRA 微调** $> 22.5\text{ GB}$ (单张 24GB 卡勉强)
- ▶ **全参微调** $> 70\text{ GB} \Rightarrow \text{A100-80G} / \text{H100}$
- ▶ **时延:** VLM 只前向一次 (缓存), 10 步只跑 300M 专家

SmolVLA (450M)

- ▶ **参数:** SmolVLM-2 (上半层跳过) + 动作专家 $\sim 100\text{M}$
- ▶ **权重** (bf16) $\approx 0.9\text{ GB}$, 下载约 1 GB
- ▶ **推理** $\approx 1\sim 2\text{ GB}$ (notebook 实测 $\sim 1\text{ GB}$) $\Rightarrow 8\text{GB}$ 卡无压力, CPU / MacBook 也能跑
- ▶ **微调:** 单张消费级 GPU / Colab 即可
- ▶ **时延:** 比 π_0 快 $\approx 40\%$ 、内存省 6~7 倍

三条经验法则 (拿它估任何 VLA)

推理: bf16 权重 = 2 B/参数 + 激活 (多相机视觉是大头)。 **全参微调** (Adam) $\approx 16\text{ B/参数} \Rightarrow 3.3\text{B}$ 光权重态就 $\approx 53\text{ GB}$, 加激活破 70 GB。 **LoRA:** 冻结主干、只训低秩适配器 $\Rightarrow$ 接近推理开销。 (代码里 `from_pretrained` 后设 `device="mps"/"cpu"` ——SmolVLA 真能在 Mac 上跑通。)



对照表: π_0 vs SmolVLA

	π_0	SmolVLA
总参数	3.3B	450M
VLM 骨干	PaliGemma (Gemma 2.6B)	SmolVLM-2 (SigLIP + SmolLM2)
动作专家	300M	$\sim 100\text{M}$ ($d = 0.75 d_{\text{VLM}}$)
专家间交互	分块因果 自注意力	因果 SA + 交替交叉注意力
预训练数据	1000 万 + 轨迹 ($\approx 9.1\%$ 开放)	2 万 + 轨迹 (100% 社区开放)
视觉输入	3 路相机 (缺失用掩码槽)	每帧 64 token、跳过上半 VLM
推理	10 步流积分 + K/V 缓存	10 步流积分 + K/V 缓存 + 异步栈
资源	>8 GB 显存	CPU 可推理; 省内存 6–7 倍、快 $\approx 40\%$

一句话

同一张配方 (VLM 骨干 + 流匹配动作专家 + 动作分块), 两种哲学: π_0 用**规模**换上限, SmolVLA 用**工程**换可及性。

代码：在 SO-100 上运行 π_0 / SmolVLA



```
from lerobot.policies.smolvla.modeling_smolvla import SmolVLAPolicy
# pi0 同理: from ...pi0.modeling_pi0 import PIOPolicy
from lerobot.policies.factory import make_pre_post_processors
from lerobot.policies.utils import build_inference_frame, make_robot_action

model_id = "lerobot/smolvla_base" # 或 "lerobot/pi0_base"
model = SmolVLAPolicy.from_pretrained(model_id)
preprocess, postprocess = make_pre_post_processors(model.config, model_id)

# 相机 key 必须与训练时一致! (见 Hub 模型卡 info.json)
# SmolVLA 期望 camera1/2/3; pi0 期望 base_0_rgb/left_wrist_0_rgb/
# right_wrist_0_rgb
camera_config = { # 只给 2 路, 第 3 路缺失会被自动 mask
    "camera1": OpenCVCameraConfig(index_or_path=0, width=640, height=480,
                                    fps=30),
    "camera2": OpenCVCameraConfig(index_or_path=1, width=640, height=480,
                                    fps=30)}
robot = SO100Follower(SO100FollowerConfig(
    port=follower_port, id=follower_id, cameras=camera_config))
robot.connect()

task = "pick the red block"
for _ in range(MAX_STEPS):
    obs = robot.get_observation()
    frame = build_inference_frame(obs, dataset_features,
                                  device, task=task)
    action = model.select_action(preprocess(frame))
    action = make_robot_action(postprocess(action), dataset_features)
    robot.send_action(action)
```

逐段解读

- ▶ from_pretrained: 从 Hub 拉取预训练 VLA——**换模型只改两行** (import 与 model_id)
- ▶ 预 / 后处理器负责归一化、设备搬运 (CUDA / MPS / CPU 均可)
- ▶ **相机 key 必须匹配训练配置**: π_0 用 base_0_rgb + 左右腕; SmolVLA 用 camera1/2/3, 缺的**自动 mask**
- ▶ select_action: 内部完成**图像过 VLM + 语言注入 + 10 步流积分** (正是前面结构图的一次前向)
- ▶ device 可设 cuda / mps / cpu——SmolVLA 连 **MacBook** 都能推理
- ▶ 语言指令 task 就是普通字符串——这就是「通才」的接口

完整脚本见教材 snippets/ch5/01_using_pi0.py 与 02_using_smolvla.py.



第 5 节

规模、显存与前沿



模型有多大？显存要多少？——速查表

据各官方 README / 论文 / 模型卡 (bf16 权重 ≈ 2 字节/参数；信息截至 2026 年初)：

模型	参数量	推理显存	微调显存	参考硬件
ACT (十二讲实测)	52M	~ 0.3 GB	bs8 ≈ 3.4 GB	任意 6GB+ 卡
Diffusion Policy	0.1–0.3B	2–4 GB	24 GB 单卡	RTX 3060+
Octo	27/93M	< 1 GB	24 GB 单卡	几乎任意 GPU
SmoIVLA	450M	$\sim 1\text{--}2$ GB (CPU 也可)	单消费级 GPU / Colab	RTX 3060、MacBook
GR00T N1.5	$\sim 2\text{--}3$ B	$\sim 8\text{--}10$ GB	LoRA: 4090; 全参: H100	Jetson Orin 可部署
π_0	3.3B	> 8 GB	LoRA > 22.5 GB; 全参 > 70 GB	4090 (推理 + LoRA)
OpenVLA	7B	~ 15 GB bf16 (int4 ~ 7 GB)	LoRA: $1 \times A100$; 全参: $8 \times A100$	4090 bf16 ≈ 6 Hz

显存经验法则（第十二讲公式的放大版）

推理：bf16 权重 2 B/参数 + 激活与 KV 缓存 1–3 GB。 **全参微调 (Adam)：** ≈ 16 B/参数 (bf16 权重 + 梯度, fp32 主权重 + 两份动量) $\Rightarrow 7$ B 模型 ≈ 112 GB——这就是 OpenVLA 全参要 $8 \times A100$ 、 π_0 要 > 70 GB 的原因。 **LoRA：**冻结权重只训低秩适配器 $\Rightarrow$ 接近推理开销 + 少量。

你的 GPU 能跑什么？



硬件档位	能做的事	典型代表
8 GB 笔记本卡 (本课环境)	ACT / DP 全流程; SmolVLA 推理 + 小批微调; π_0 推理 (勉强)	RTX 4070 Laptop
24 GB 消费旗舰	π_0 LoRA 微调; OpenVLA bf16 推理; GR00T 微调	RTX 4090
80 GB 数据中心卡	π_0 全参微调; OpenVLA LoRA	A100 / H100
多卡集群	7B+ 全参微调 (FSDP); 从零预训练	8×A100 起步
嵌入式 / 端侧	SmolVLA、GR00T (TensorRT)、Gemini Robotics On-Device	Jetson Orin / Thor

量化：能省，但有代价

OpenVLA 实验：int8 / int4 推理**任务成功率基本保持**，int4 显存 ~ 7 GB；但在 4090 上 int4 反而**更慢**（反量化开销）——省显存 $\neq$ 提速度。

课程视角

这门课的完整管线（采数据 $\rightarrow$ 训 ACT/DP $\rightarrow$ 微调 SmolVLA $\rightarrow$ 部署）在**一张消费级 GPU** 上就能走通——这在 RT-2 时代（55B、闭源）是不可想象的。



前沿速览（一）：闭源巨头在做什么

模型	机构 / 时间	解决了什么
Gemini Robotics (+ER) Helix	GDM · 2025.03 Figure AI · 2025.02	把前沿多模态 推理 带入 VLA (折纸、密封袋); ER 专攻具身空间理解 双系统 : S2 (7B VLM, 7-9 Hz) + S1 (80M, 200 Hz) $\Rightarrow$ 35 自由度人形上身全控制, 双机器人零样本协作, 全部跑在 机载 GPU
$\pi_{0.5}$	PI · 2025.04	同一模型先出 语义子任务 再出动作 (分层) $\Rightarrow$ 开放世界泛化 : 打扫从未见过的真实厨房 / 卧室
Gemini Robotics 1.5	GDM · 2025.09	「 思考后行动 」+ 跨形态技能迁移 (ALOHA $\leftrightarrow$ 人形 $\leftrightarrow$ 双臂 Franka); ER 1.5 做智能体编排
$\pi_{0.6}^*$ + RECAP	PI · 2025.11	真实世界 RL 后训练 (优势条件化 + 自主经验 + 人工纠正): 吞吐 $\times 2$ 、失败率减半, 连续数小时无人值守做咖啡 / 叠衣

两条主线清晰可见：**分层 / 双系统架构**（慢思考 + 快控制）与 **RL 后训练**走上真机。

前沿速览（二）：开源阵营与中国力量



模型	机构 / 时间	要点
GR00T N1 / N1.5	NVIDIA ·2025	2.2B 开源 人形 基础模型：Eagle VLM（系统 2）+ DiT 流匹配（系统 1, 120 Hz）；「数据金字塔」：真机 + 合成（DreamGen 世界模型造数据）+ 网络视频
GO-1	智元 AgiBot ·2025.03	ViLLA 架构（VLM + 隐动作规划器 + 动作专家）；配套开放 AgiBot World （百台机器人、百万轨迹）
GR-3	字节 Seed ·2025.07	4B：Qwen2.5-VL + DiT 流匹配头；用 VR 人类轨迹 做廉价 few-shot 适配；长程柔性物体任务（挂衣服）
RDT-1B / RDT-2	清华·2024–25	1.2B 扩散 Transformer 双臂操作；RDT-2 用 UMI 手持夹爪 野外数据，零样本换形态
WALL-OSS / Galaxea G0 等	中国多家·2025	推理-动作紧耦合（CoT→ 动作）、双系统 + 开放数据集—— 开源权重的主力军
GEN-0	Generalist AI ·2025.10	10B+，宣称 27 万 + 小时 真实操作数据与具身 scaling laws

注：本页信息截至 2026 年初；该领域以**月**为单位迭代，请以各家最新发布为准。

2025-26 的六大趋势



(1) 配方收敛

VLM 骨干 + 流匹配动作专家 (2-7B) 成为事实标准: π_0 、GR00T、GR-3、SmolVLA 全在此列; 离散 token 路线靠 FAST 类动作分词器存续。

(2) 双系统 / 分层成为默认

慢的「想」(VLM, 个位数 Hz) + 快的「动」(小模型, 百 Hz): Helix S1/S2、GR00T、 $\pi_{0.5}$ 子任务 → 动作、Gemini 的 ER 编排。

(3) RL 后训练上真机

RECAP/ $\pi_{0.6}^*$ 证明真实部署中 RL 微调可行——预训练 BC 给底子, RL 给可靠性。(呼应前十讲: RL 没有离场, 而是换了位置。)

(4) 端侧化

Gemini Robotics On-Device、SmolVLA、Jetson 上的 GR00T——离线、低延迟、隐私友好。

(5) 数据多元化

遥操作之外: 人类视频、VR 示范 (GR-3)、UMI 手持设备 (RDT-2)、世界模型合成数据 (DreamGen、V-JEPA 2); GEN-0 宣称具身 scaling laws。

(6) 思考后行动

具身推理轨迹 (CoT) 进入 VLA: Gemini Robotics 1.5、WALL-OSS——「智能体」与「策略」的边界在消融。

本讲总结

1 流匹配：扩散的连续确定性推广

学一个速度场 v_θ 把噪声推成数据； π_0 用最简单的直线插值

$z_t = (1 - t)z_0 + tz_1$ ，目标速度恒为 $z_1 - z_0$ ，MSE 回归即训练；推理 = 10 步 Euler 积分——实时控制的关键。

2 通才范式：预训练-再适配

RT-1 规模化 → RT-2 创立 VLA（动作并入 token）→ Open-X/DROID 数据基建 → OpenVLA 开源平权；趋势：数据更大更去中心化，模型 55B → 450M。

3 现代 VLA: π_0 与 SmolVLA

统一 Transformer + 分工专家；VLM 管看与听，动作专家用流匹配生成连续动作块； π_0 ：分块因果注意力、Beta 时刻采样、 π 数据集预训练 + 微调；SmolVLA：450M、社区数据、CA/SA 交替、砍半 VLM、异步推理。

4 规模与前沿

推理 2B/参数、全参微调 16B/参数、LoRA 居中；消费级 GPU 即可走通全管线；前沿：双系统、RL 后训练、端侧化、数据多元化。

全课程的一条线

反馈 $u = \pi(x)$ （十一讲）→ RL 学反馈（前十讲）→ 示范锚定行为（十二讲）→ 生成式策略治多模态（十二、十三讲）→ **流匹配 + VLM 先验 + 大数据 = 通才策略**（本讲）。

没解决的问题

可靠性仍低于工业标准；长尾物理常识缺失；评测难以标准化；数据规模距 LLM 尚差几个数量级——这门课结束的地方，正是这个领域开始的地方。

动手建议

用 SO-100 采 50 条示范 → 微调 `lerobot/smolvla_base` → 与你训过的 ACT 对比成功率——一张消费级 GPU 足矣。





谢谢!

深圳技术大学

C-4 教室